Series de Pandas

Estadística con Python

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

Series de Pandas

Pandas

- *Pandas* es una biblioteca de Python ampliamente utilizada para el análisis y procesamiento de datos.
- Proporciona estructuras de datos rápidas, flexibles y útiles para cálculo estadístico y ofrece funciones que facilitan la entrada, organización y transformación de datos.
- Las dos principales estructuras de datos de Pandas, `Series` y `DataFrame`, agotan la gran mayoría de los casos de uso típicos en estadística.

Series de pandas

Nota

Una *Serie* de Pandas es un arreglo unidimensional etiquetado (`ndarray`). Este objeto, que admite la indexación basada en enteros y etiquetas, proporciona una gran cantidad de métodos para realizar operaciones relacionadas con el índice.

Creación de Series

Creación Series de Pandas

- A continuación se muestra como crear Series de Pandas.
- Lo primero es importar el paquete `pandas`, lo hacemos con el nombre `pd` como es costumbre entre la comunidad de usuarios de esta biblioteca.

```
import pandas as pd
import numpy as np
datoS=pd.Series([22,15,19])
datoS
```

```
0    22
1    15
2    19
dtype: int64
```

De manera predeterminada, la función `Series()` asigna índices numéricos cuando, al momento de crear la serie, no se entregan las etiquetas, como ocurre con los indices de un arreglo de `numpy` o de una lista.



Métodos para exploración de series

Exploración inicial: métodos `.head` y `.tail`

- Cuando trabajamos con Series que contienen una gran cantidad de datos, suele ser poco práctico imprimirlos todos en pantalla.
- Para una inspección rápida, usamos los métodos `.head(n)` y `.tail(n)`, los cuales permiten extraer los primeros y últimos `n` elementos de la Serie, respectivamente.
- Si no se especifica el valor de `n`, por defecto ambos métodos devuelven 5 elementos.

Ejemplo de uso de `.head` y `.tail`

```
# Creamos una serie más larga para el ejemplo
datos = pd.Series([12, 45, 32, 18, 56, 29, 78, 91, 14, 63, 88])

datos.head()
```

```
0    12
1    45
2    32
```



Recuperación de datos de una serie

Recuperación de valores de la Serie

Como ocurre con los diccionarios, podemos recuperar los indices de una serie mediante el método `.index` y los valores con el método `.values`

```
edadS.index    ①
edadS.values   ②
```

① Índices de la Serie.
② Valores de la Serie.

```
Index(['Luis', 'Brenda', 'Diego'], dtype='object')
array([15, 22, 19])
```

Recuperación de valores de la Serie

- Observe que los valores de la serie se recuperan como un arreglo de `NumPy`.
- La recuperación de datos de una serie se hace de manera análoga a como se hace con los arreglos o los diccionarios
  - Por indice,



Modificación del índice

Asignación futura del índice

- Es posible modificar o establecer las etiquetas de una Serie después de haberla creado, reasignando una nueva lista al atributo `.index`.
- **¿Por qué es útil?** Es muy común que los datos numéricos provengan de un archivo o medición y carezcan de etiquetas iniciales . Posteriormente, obtenemos las etiquetas desde otra fuente y necesitamos integrarlas.
- **Regla fundamental:** La cantidad de etiquetas nuevas proporcionadas debe ser exactamente igual a la cantidad de elementos en la Serie.

Ejemplo de asignación del índice

```
# 1. Cargamos datos que inicialmente no tienen etiquetas descriptivas
temperaturas = pd.Series([28.5, 30.2, 29.8])
temperaturas.index

# 2. Obtenemos los códigos de las estaciones de monitoreo desde otra fuente
codigos_estaciones = ['Est_Norte', 'Est_Centro', 'Est_Sur']
```

Agregar datos a una serie

Ejemplo: agregar un elemento

Agregamos un nuevo individuo a la serie edadS

```
edadS["Pedro"]=16

edadS.tail(3)
```

```
Brenda    22
Diego     19
Pedro     16
dtype: int64
```

Agregar múltiples elementos a una Serie

Para agregar **múltiples** elementos nuevos al mismo tiempo, debemos crear una nueva Serie con los datos a ingresar y luego unirlas utilizando la función `pd.concat()`.

```
nuevos= pd.Series({'Ana': 20, 'Juan': 25, 'Sofia': 21})
```

Conversión de tipos

Modificación del tipo de dato

- En ocasiones, los datos de una Serie no tienen el formato adecuado para el análisis. Por ejemplo, números que fueron importados por el sistema como texto (`string`).
- El método `.astype()` permite forzar la conversión de los elementos de una Serie a un tipo de dato (`dtype`) específico, como enteros (`int`) o decimales (`float`).
- Es importante recordar que este método devuelve una **nueva** Serie con los tipos cambiados; no modifica la original a menos que se sobreescriba.

Ejemplo de modificación de tipo

Serie donde los números están almacenados como texto

```
texto = pd.Series(["15.5", "20.1", "18.3"])
texto.dtype
```

```
dtype('O')
```

Conversión a números de punto flotante (decimales)

```
numerico = texto.astype(float)
```

Alineación automática de índices

Alineación automática de índices en operaciones

- Una de las características más potentes de Pandas es que, al realizar operaciones matemáticas entre dos Series, los datos se **alinean automáticamente basándose en sus etiquetas**, no en su posición.
- Si una etiqueta está presente en una Serie pero no en la otra, Pandas introduce un valor faltante (`NaN`) en el resultado para esa etiqueta.
- Esto evita errores graves (que sí ocurrirían con listas normales) cuando se cruzan conjuntos de datos desordenados o con diferente cantidad de registros.

Ejemplo: Alineación automática de índices

```
# Notas del primer y segundo corte de un grupo de estudiantes
corte1 = pd.Series({'Luis': 4.0, 'Brenda': 4.5, 'Diego': 3.8, 'Ana': 4.2})
corte2 = pd.Series({'Brenda': 4.6, 'Ana': 4.0, 'Diego': 4.1, 'Pedro': 3.5})

# Sumamos ambas series para obtener el acumulado
suma= corte1 + corte2
```

Datos faltantes en una Serie

Datos faltantes en series de Pandas

- Las series de Pandas tienen la propiedad de poder tratar con datos faltantes,
- Los cuales se representan como: `"NA" = Not available`, `"NaN" = Not a number`, `None = Null/nonetype object`.
- A continuación se crean series que tienen datos faltantes

Datos faltantes en series de Pandas

```
edadSN=pd.Series([21, 15, None], index=['A', 'B', 'C'])          ①
edadSN=pd.Series([21, 15,np.nan],index=['A', 'B', 'C'])          ②
edadDic={'A':12, 'B':15, 'C':None}                               ③
edadSN=pd.Series(edadDic)                                        ④
edadSN=pd.Series(edadDic,index=["A","B","D"])                    ⑤
edadSN
```

① Serie creada a partir de una lista con un valor en `None`.
② Serie creada a partir de un arreglo de Numpy con un valor en `nan`.
Diccionario con una clave cuyo valor es `None`.